

Trabalho Avaliativo de Inteligência Artificial

Implementação e comparação de UCS, Busca Gulosa e A* — Problema 2: Torre de Hanói

Disciplina: Inteligência Artificial **Professor:** Marcos Augusto Campagnaro Mucelini
Integrantes: Kauã Cardoso Benini, Gabriel Benini, Felipe Donatoni **Data:** 17/09/2026

1. Introdução

O grupo escolheu o **Problema 2 — Torre de Hanói**. O motivo principal da escolha foi a vantagem que esse problema tem sobre os demais da lista: a solução ótima é conhecida por uma **fórmula matemática** ($2^n - 1$ movimentos, para n discos), o que dá um gabarito exato para conferir se a implementação da busca está correta — em vez de confiar apenas visualmente no resultado. Isso permitiu validar os três algoritmos de forma objetiva em cada teste.

2. Modelagem

Item	Modelagem
Estado	tupla de N posições, uma por disco (disco 0 = o menor). O valor em cada posição é o pino (0=A, 1=B, 2=C) onde aquele disco está. Ex.: (0, 0, 2, 1) para 4 discos
Ações	mover o disco do topo de um pino para outro. O "topo" de um pino é sempre o disco de menor índice presente nele, já que um estado válido nunca tem disco maior sobre um menor
Custo	1 por movimento
Objetivo	todos os discos no pino C (índice 2)
Espaço de estados	3^n para n discos

Usamos tupla, e não lista, pelo mesmo motivo de sempre: o estado precisa ser *hashable* para entrar no dicionário de "estados já alcançados" da busca, que evita reexpandir a mesma posição mais de uma vez.

Particularidade do problema. Como todo movimento custa exatamente 1, e como observa a própria ficha do problema, a UCS aqui se comporta como **busca em largura**: sem heurística, ela expande os estados em ordem de profundidade crescente, porque todos os caminhos de mesmo número de movimentos têm o mesmo custo acumulado.

3. Heurística

Usamos a heurística sugerida na própria ficha do problema: **número de discos que ainda não estão no pino destino**.

Por que é admissível. Cada disco que ainda não está no pino destino vai precisar de **pelo menos um movimento** antes do fim da solução — ele precisa sair de onde está e chegar ao destino em algum momento. Logo, o número de movimentos que ainda faltam é sempre maior ou igual ao número de discos fora do lugar: a heurística nunca superestima o custo restante.

Por que é uma heurística fraca (de propósito). Ela não captura a estrutura recursiva do problema: um disco pode estar "fora do lugar" e precisar de 1 movimento, ou pode estar preso sob vários discos maiores e precisar de muitos movimentos antes de sequer poder se mover. A heurística trata os dois casos como iguais (conta 1 para cada disco fora do lugar, independentemente de quão "enterrado" ele está), por isso ela discrimina mal entre estados — efeito visível na Seção 4.

4. Resultados

Testamos com 3 a 10 discos, como sugerido na ficha do problema, limite de tempo de 60s (a busca mais lenta, com 10 discos, levou menos de 1 segundo).

Discos	Gabarito (2^n-1)	UCS: nós exp.	Gulosa: nós exp.	A*: nós exp.	Custo (os 3)
3	7	24	17	17	7
4	15	70	59	53	15
5	31	232	179	177	31
6	63	686	622	585	63
7	127	2.144	1.786	1.865	127
8	255	6.390	6.267	5.865	255
9	511	19.512	17.642	18.201	511
10	1.023	58.366	62.080	55.881	1.023

Em todos os 24 testes (8 valores de $n \times 3$ algoritmos), o custo encontrado bateu exatamente com o gabarito da fórmula 2^n-1 — a primeira confirmação de que a implementação está correta.

A **explosão combinatória** fica clara na coluna de nós expandidos: cada disco adicional multiplica o número de nós por um fator de aproximadamente 3, coerente com o espaço

de estados de tamanho 3^n . Testamos informalmente até 12 discos fora do benchmark oficial: a UCS levou mais de 23 segundos, contra cerca de 12–17 segundos de Gulosa/A* — a diferença entre busca cega e busca informada cresce junto com n .

Um resultado que não esperávamos: a Gulosa nem sempre expandiu menos nós que o A* (em $n=9$, 17.642 contra 18.201 do A*; mas em $n=10$, o inverso: 62.080 contra 55.881). Como a heurística é fraca, ela guia pouco a busca, e às vezes o custo extra do A* de calcular $g(n)+h(n)$ não é compensado por uma economia real de nós — diferente do que vimos no problema de xadrez que consideramos inicialmente, onde a heurística era mais informativa e a vantagem do A* era mais consistente.

5. Discussão

1) Algum algoritmo devolveu uma solução pior que os outros? Qual, e por quê? Não — nos 8 valores de n testados, os três algoritmos sempre encontraram exatamente o custo ótimo (2^n-1), inclusive a Gulosa. Isso acontece porque, apesar de a heurística ser fraca, ela nunca chega a "enganar" a Gulosa o suficiente para levá-la por um caminho mais caro dentro do espaço de estados testado — a estrutura muito regular e simétrica da Torre de Hanói (o espaço de estados é uma árvore quase completa) não oferece atalhos falsos como aconteceria em problemas com heurísticas mais específicas por peça, como percebemos ao comparar com o xadrez.

2) Qual expandiu menos nós? Isso significa que ele é o melhor? Na maioria dos casos, A* e Gulosa expandiram bem menos nós que a UCS (por exemplo, em $n=8$: 5.865 e 6.267 contra 6.390). Mas a diferença entre Gulosa e A* variou — ora um venceu, ora o outro (Seção 4). Isso reforça que "menos nós" não é garantia de nada sozinho: aqui não houve prejuízo de qualidade porque a Gulosa nunca errou o custo, mas essa garantia não vem da Gulosa em si (ela não garante otimalidade por definição) — vem de uma característica deste problema específico.

3) A heurística de vocês é admissível? Como sabem? Sim — todo disco fora do pino destino precisa de pelo menos 1 movimento antes do fim da solução, então contar quantos discos estão fora do lugar nunca supera o número real de movimentos que faltam. Na prática, confirmamos isso porque o A* nunca divergiu do gabarito conhecido (2^n-1) em nenhum dos 8 testes — se a heurística superestimasse, isso poderia ter acontecido.

4) Se vocês relaxassem mais regras do problema, a heurística ficaria melhor ou pior? Por quê? Ficaria **ainda mais fraca**. A heurística atual já ignora a estrutura recursiva do problema (o fato de um disco poder estar "enterrado" sob outros); se relaxássemos mais — por exemplo, ignorando também a regra de que um disco maior não pode ficar sobre um menor — perderíamos até a noção de quais movimentos são válidos, e a heurística deixaria de fazer sentido. Por outro lado, uma heurística **mais forte** (não pedida no trabalho) poderia calcular recursivamente quantos movimentos os $n-1$

discos menores ainda precisam para se reorganizar, capturando melhor a estrutura do problema — mas isso já seria quase resolver o problema, não apenas estimar.

5) Houve empate entre algoritmos? O que isso revela sobre o problema? Sim, empate em **custo** em absolutamente todos os testes. Isso revela que a Torre de Hanói, com custo uniforme e um espaço de estados muito regular, é um problema em que a qualidade da solução não é o que diferencia os três algoritmos — a diferença aparece só na eficiência (nós expandidos), e ainda assim de forma modesta, porque a heurística sugerida é fraca. É um contraste interessante com problemas de heurística mais forte, em que o A* costuma ter uma vantagem de eficiência muito mais consistente.

6) Em que situação real vocês usariam cada um dos três? - **UCS:** quando não há nenhuma heurística disponível, mas o custo do caminho precisa ser garantidamente ótimo — por exemplo, um planejador que só tem acesso ao custo bruto das ações, sem informação adicional sobre o problema. - **Busca Gulosa:** quando uma resposta rápida e "razoável" já basta, e o risco de uma solução subótima é aceitável — por exemplo, sugestões automáticas em tempo real que podem ser refinadas depois. - **A*:** quando existe uma heurística admissível, mesmo que fraca, e se quer a garantia de otimalidade com alguma economia de esforço — por exemplo, planejadores de movimento robótico que precisam reorganizar objetos empilhados (um problema com a mesma "forma" da Torre de Hanói) sem descartar a garantia de um plano de custo mínimo.

6. Conclusão

O trabalho confirmou, com um gabarito matemático exato à disposição ($2^n - 1$), que os três algoritmos implementados a partir da mesma função de busca produzem sempre a solução ótima correta. Também mostrou, de forma nítida, a explosão combinatória do espaço de estados (3^n) ao variar o número de discos, e como uma heurística admissível, mesmo fraca, ainda traz alguma economia de nós expandidos frente à busca cega — embora de forma menos consistente do que uma heurística mais informativa traria. O principal aprendizado foi perceber que uma heurística pode ser correta (admissível) e ainda assim contribuir pouco, dependendo de quanto ela realmente captura da estrutura do problema.

Extra: Interface gráfica (opcional, +10%)

Implementamos a interface gráfica opcional em `gui.py`, usando apenas `tkinter` (biblioteca padrão do Python). Ela desenha o tabuleiro com os pinos e discos, anima a solução encontrada passo a passo — destacando visualmente o disco em movimento a cada lance — e exibe um gráfico de barras comparando os nós expandidos por UCS, Gulosa e A* para o número de discos escolhido, atualizado a cada nova resolução. O usuário escolhe o número de discos (3 a 10), o algoritmo a animar, e a velocidade da animação.

7. Referências

RUSSELL, S. J.; NORVIG, P. *Artificial Intelligence: a Modern Approach*. 4. ed. Global Edition. Pearson, 2021. Capítulo 3, Seções 3.3, 3.5 e 3.6.

Slides das Aulas 04 e 05 da disciplina de Inteligência Artificial, Prof. Marcos Augusto Campagnaro Mucelini.

Anexo: código-fonte

Ver arquivos `hanoi.py`, `busca.py`, `resolver.py`, `main.py` e `gui.py` anexados ao `.zip` de entrega.